PilotFish Static File Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This step **replaces the transaction body** with the contents of a file (or an XML listing of a folder) from disk — like dropping a template or test payload into the route. You can optionally delete or move the source file after it is read.

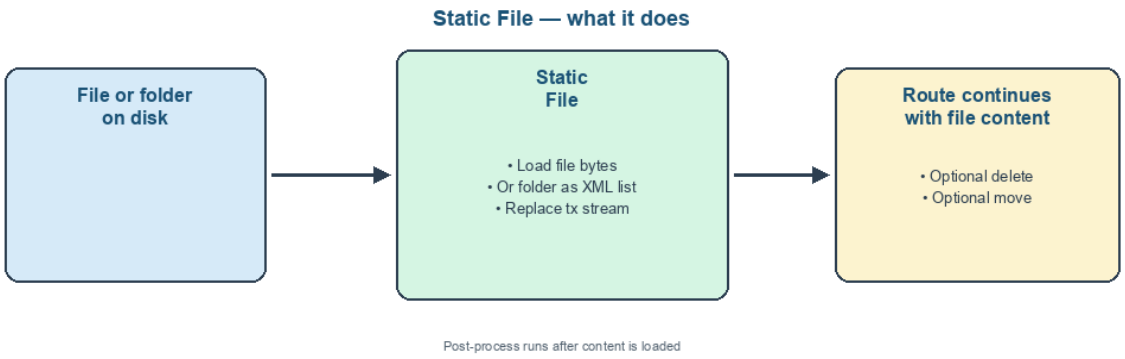

Figure 1: At a glance (plain English)

- **Loads** a single file's bytes into the transaction stream.
- **Or** converts a **directory** listing to XML via `XMLUtil.filesToXMLStream`.
- **Optionally deletes** or **moves** the source after successful read.
- **Closes** the prior stream before replacing it.

Purpose

Deep dive into **Static File** (`StaticFileProcessor`) in **`eip.war.hs.26R1.11`**.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-file-1.0.1.jar`
Implementation class	`com.pilotfish.eip.modules.file.StaticFileP`
Type string	`Static File`
Help ID	`console.processor.staticfile`
Categories	`FILES`

---

1. Conceptual model

```
Configured path (file or directory)
        |
        v
  Static File Processor
        |
        +-- file  -> copy bytes to EIP cache -> setDataStream
        +-- dir   -> XMLUtil.filesToXMLStream(listing) -> setDataStream
        |
        +-- post-process: Do Nothing | Delete | Move
        |
        v
  Route continues with new stream
```

---

2. High-level behavior (`processData`)

1. Resolve **Target directory** when post-process is **Move**; throw if path is not a directory.
2. Resolve **File/Folder name** via `manager.getFile("FileName", data, loader)`.
3. If path is **null**, return `data` unchanged (no error).
4. **Regular file** (`exists && !isDirectory`):
   - Close existing `data.getDataStream()`.
   - Cache sized to `file.length()`; copy via `FileInputStream`.
   - `data.setDataStream(cache.getInputStream())`.
5. **Directory** (`exists && isDirectory`):
   - Close stream; build XML listing of `listFiles()` into default cache.
   - `includeContents` flag to `filesToXMLStream` is **`true`**.
6. **Missing path** -> `EIPException("Specified file does not exist : ...")`.
7. **Move** post-process:
   - File: `FileUtils.copyFileToDirectory` then `recursivelyDelete` source.
   - Directory: `copyDirectoryToDirectory` then `recursivelyDelete`.

- I/O failures **logged** at error level — **not rethrown**.

8. **Delete** post-process: `recursivelyDelete` — failures logged, not rethrown.
9. Return `data`.

---

3. Configuration reference

3.1 File/Folder name

Property	Value
UI label	File/Folder name
Tag	`FileName`
Type	Path
Default	empty
Required	yes (descriptor)
Tooltip	Specifies the file whose contents will be used to replace the transaction data

Supports file **or** directory paths relative to route resources / expression resolution.

3.2 Postprocess operation

Property	Value
UI label	Postprocess operation
Tag	`PostProcessOperation`
Type	Multiple choice
Choices	`Do Nothing`, `Delete`, `Move`
Default	`Do Nothing`

3.3 Target directory

Property	Value
UI label	Target directory
Tag	`TargetDirectory`
Type	Path (directory)
Visible when	Postprocess = **Move**
Writeability	checked
Tooltip	The directory for the file to be moved to.

Validated at runtime when Move is selected — must exist as a directory before read.

3.4 Inherited AbstractProcessor

Tag | `ExecuteProcessor` | Default `true` |

---

4. Transaction attributes

None written by this processor.

---

5. Worked examples

5.1 Template injection

File `/data/templates/ack.xml`, Postprocess **Do Nothing** — every transaction body becomes the template file bytes.

5.2 Pickup-and-move

Inbound file path from listener attribute, Postprocess **Move** to `/data/processed/` — content loaded then source removed after copy (errors logged only).

5.3 Directory manifest

Folder `/data/batch/` — stream becomes XML describing contained files (with contents per `filesToXMLStream(..., true)`).

---

6. Known quirks and caveats

1. **Prior stream closed** — upstream must not rely on the old stream handle.
2. **Move/delete errors swallowed** — monitor transaction logs for move/delete failures.
3. **Null FileName** — silent no-op.
4. **Directory XML shape** — defined by `XMLUtil.filesToXMLStream`, not raw concatenation.
5. **Move uses copy-then-delete** — not atomic rename across volumes.

---

7. Operational recommendations

Goal	Guidance
Test data	Use Do Nothing against fixed route resources
File pickup	Prefer Directory Listener; use Static File for explicit injection
Move failures	Add verification stage or use Directory Listener post-process

Goal	Guidance
Large files	Consider cache/memory impact of full-file read

---

8. Source index

Topic	Path
Static File processor	`com/pilotfish/eip/modules/file/StaticFileProcesso`
Module JAR	`xcs-modules-file-1.0.1.jar`

Deep-dive – Static File Processor – 26R1.11 – August 2026.